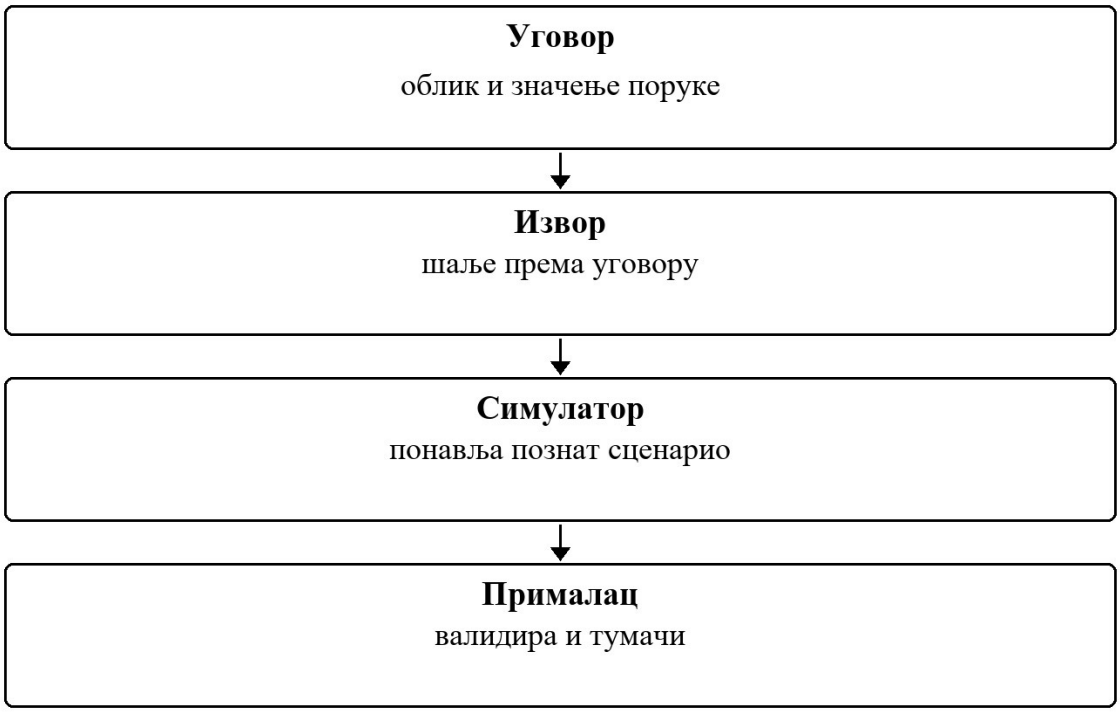
Уговори, симулатори и поновљивост

Основе дистрибуираног програмирања · 02

Компоненте могу да сарађују само ако деле уговор о облику поруке и очекиваном понашању. Уговор није споредна документација: он је граница која омогућава независну промену и даје јасан одговор на питање да ли порука има значење за примаоца.

Симулатор даје тиму поновљив начин да провери токове без ослањања на реалну опрему, нестабилну мрежу или случајне улазе. Поновљивост је услов да се грешка разуме и да поправка заиста буде доказана.



Слика 1. Заједнички језик компоненти и безбедан начин учења из грешке

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

2.1 Порука мора да има заједничко значење

- Објављена верзија уговора чува очекивања постојећих корисника.
- Валидација одбија поруку коју систем не разуме, уместо да јој нагађа значење.
- Симулатор одваја стварни проблем система од случајности спољног света.

Питање за дискусију

Коју поруку бисте најпре претворили у стабилан уговор и који сценарио симулатора би најбоље открио неспоразум између пошиљаоца и примаоца?

2.2 Како се ова тема види у развоју

Тим записује мали скуп порука и сценарија који представљају кључне токове. Затим покреће исти сценарио више пута, са познатим улазима, и проверава да сваки пут добија исти релевантан резултат.

Честе грешке које треба препознати

- Промена поруке без верзије и без размишљања о примаоцима.
- Симулатор који садржи пословну логику централног система.
- Тест који зависи од случајног времена или спољне услуге.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

2.3 Пример: прималац не разуме нову поруку

Пошиљалац проширује поруку новим пољем и сматра да промена нема последице. Старији прималац, међутим, другачије тумачи вредност или одбацује поруку. Уговор чува обе стране тако што облик, верзију и правила компатибилности чини експлицитним пре него што порука стигне до продукционог тока.

- Верзија говори примаоцу како да тумачи поруку.
- Симулатор може поновити исти ток са старим и новим примаоцем.
- Нејасна или непозната порука добија контролисан исход, а не произвољно тумачење.

Како се сценарио анализира

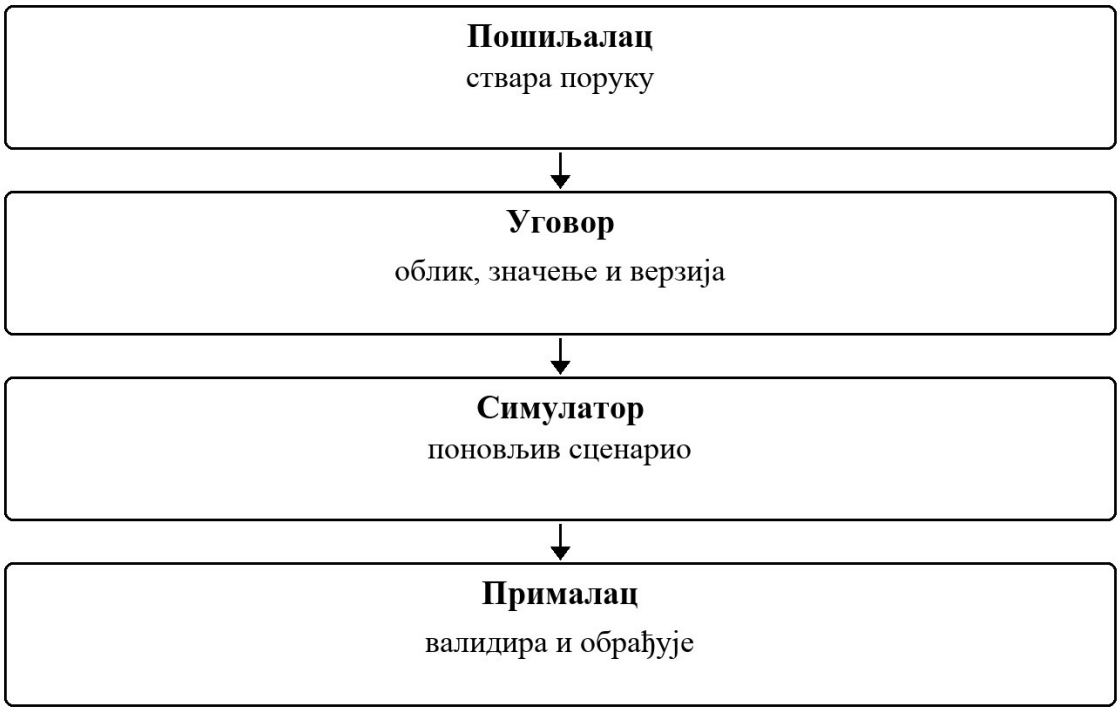
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

2.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

2.5 Шема уговора између компоненти



Слика 2. Шема уговора између компоненти

Стабилан уговор омогућава компонентама да се мењају без нагађања о значењу поруке.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

2.6 Упоредни преглед елемената тока

Табела 1. Елемент и Улоīа у ток̄у

Елемент	Улога у току	Питање које треба поставити
Уговор	облик и значење поруке	Шта се дешава када уговор није доступан или даје застарелу информацију?
Извор	шаље према уговору	Шта се дешава када извор није доступан или даје застарелу информацију?
Симулатор	понавља познат сценарио	Шта се дешава када симулатор није доступан или даје застарелу информацију?
Прималац	валидира и тумачи	Шта се дешава када прималац није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

2.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Објављена верзија уговора чува очекивања постојећих корисника.
2. Валидација одбија поруку коју систем не разуме, уместо да јој нагађа значење.
3. Симулатор одваја стварни проблем система од случајности спољног света.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Коју поруку бисте најпре претворили у стабилан уговор и који сценарио симулатора би најбоље открио неспоразум између пошиљаоца и примаоца?

2.8 Извршиви пример

Пример показује две ствари које омогућавају да се компоненте развијају независно: порука има експлицитан уговор који се проверава на улазу, а удаљена станица се замењује симулатором чији је ток у потпуности одређен сценаријом. Исти сценарио увек даје исте поруке, па се и грешка може поновити.

Пример 1. Покрет̄ање демонстрације и т̄ест̄ова

```
cd examples/odp-dotnet/vezba-02-ugovori-simulator
dotnet run --project src/Odp.Vezba02.ConsoleUi
dotnet test Odp.Vezba02.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-02-ugovori-simulator/src/Odp.Vezba02.Domain/Contracts/TelemetryContractValidator.cs

```
public Result Validate(TelemetryMessage message)
{
    ArgumentNullException.ThrowIfNull(message);

    if (message.ContractVersion != TelemetryContract.Version)
        return Result.Fail(ContractErrorCodes.UnsupportedVersion);
```

```
    if (string.IsNullOrWhiteSpace(message.StationId))
        return Result.Fail(ContractErrorCodes.StationIdRequired);
    if (message.Sequence <= 0)
        return Result.Fail(ContractErrorCodes.SequenceMustBePositive);
    if (message.SignalStrengthDbm is < TelemetryContract.MinSignalDbm or >
TelemetryContract.MaxSignalDbm)
        return Result.Fail(ContractErrorCodes.SignalOutOfRange);

    return Result.Ok();
}
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-02-ugovori-simulator/tests/Odp.Vezba02.Tests/Infrastructure/ScriptedStationSimulatorTests.cs

```
[Test]
public void Emit_WithTheSameScenario_ProducesTheSameMessages()
{
    var first = new ScriptedStationSimulator(Scenario).Emit(20);
    var second = new ScriptedStationSimulator(Scenario).Emit(20);

    Assert.That(second, Is.EqualTo(first));
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba02.ConsoleUi

```
Poruke simulatora:
  #1 09:00:00  -83.0 dBm
  #2 09:00:05 -105.0 dBm
  #3 09:00:10 -105.0 dBm
  #4 09:00:15    5.0 dBm
  #5 09:00:20 -104.0 dBm
  #6 09:00:25 -100.0 dBm
  #7 09:00:30  -81.0 dBm
  #8 09:00:35    5.0 dBm

Prihvaćeno: 6, odbijeno: 2
  #4 -> signal_out_of_range
  #8 -> signal_out_of_range

Ponovljeno pokretanje daje isti ishod: True
```

Шта треба уочити

- Уговор се проверава на улазу, а свако одбијање има стабилан код.
- Исти сценарио (исти `Seed`) даје исте поруке, па се и грешка може поновити.
- Симулатор је адаптер иза порта `IStationSimulator`; правила не знају да станица није права.

Задатак уз пример

Додати у сценарио кашњење поруке (`DelayEvery`), тако да свака Н-та порука носи `MeasuredAt` старији од претходне. Увести правило уговора које такву поруку одбија новим кодом и покрити га тестом.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Уговори, симулатори и поновљивост". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

П1 Основни ентитети, уговори и симулатор

Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.